
ReflectDrive-2: Reinforcement-Learning-Aligned Self-Editing for Discrete Diffusion Driving

Huimin Wang^{*‡}, Yue Wang^{*}, Bihao Cui[†]

Pengxiang Li, Ben Lu, Mingqian Wang, Tong Wang, Chuan Tang, Teng Zhang, Kun Zhan

LiAuto

Abstract

We introduce ReflectDrive-2, a masked discrete diffusion planner with a separate action expert for autonomous driving that represents plans as discrete trajectory tokens and generates them through parallel masked decoding. This discrete token space enables in-place trajectory revision: AutoEdit rewrites selected tokens using the same model, without requiring an auxiliary refinement network. To train this capability, we use a two-stage procedure. First, we construct structure-aware perturbations of expert trajectories along longitudinal progress and lateral heading directions and supervise the model to recover the original expert trajectory. We then fine-tune the full decision–draft–reflect rollout with reinforcement learning (RL), assigning terminal driving reward to the final post-edit trajectory and propagating policy-gradient credit through full-rollout transitions. Full-rollout RL proves crucial for coupling drafting and editing: under supervised training alone, inference-time AutoEdit improves PDMS by at most 0.3, whereas RL increases its gain to 1.9. We also co-design an efficient reflective decoding stack for the decision–draft–reflect pipeline, combining shared-prefix KV reuse, Alternating Step Decode, and fused on-device unmasking. On NAVSIM, ReflectDrive-2 achieves 91.0 PDMS with camera-only input and 94.8 PDMS in a best-of-6 oracle setting, while running at 30.2 ms average latency on NVIDIA Thor.

1 Introduction

Planning errors in imitation-learned driving policies are rarely random. They concentrate along two common axes: longitudinal speed misjudgment (overshoot, under-progress, late braking) and lateral heading drift (lane deviation, clipped turns, drivable-area violations). These are the directions along which imitation learning from expert demonstrations accumulates covariate shift [Bansal et al., 2019, Codevilla et al., 2018], and they are the directions along which an in-place correction mechanism could act. A planning representation that supports structured in-place revision is therefore well-matched to the error structure of the problem. Classical modular stacks [Fan et al., 2018, Kato et al., 2018] and end-to-end planners [Bojarski et al., 2016, Hu et al., 2023, Chitta et al., 2022, Hu et al., 2022, Jiang et al., 2023a] commit to a single trajectory; autoregressive vision-language-action (VLA) planners [Kim et al., 2024, Tian et al., 2024, Sima et al., 2024] inherit sequential decoding and revise emitted tokens only by re-rolling the full sequence; continuous diffusion planners [Janner et al., 2022, Chi et al., 2023, Liao et al., 2025, Xing et al., 2025] parallelize generation but reverse a Gaussian corruption process rather than the structured failure modes of a trained driver. Masked

^{*}Equal Contribution.

[†]Project Lead.

[‡]wanghuimin1@lixiang.com

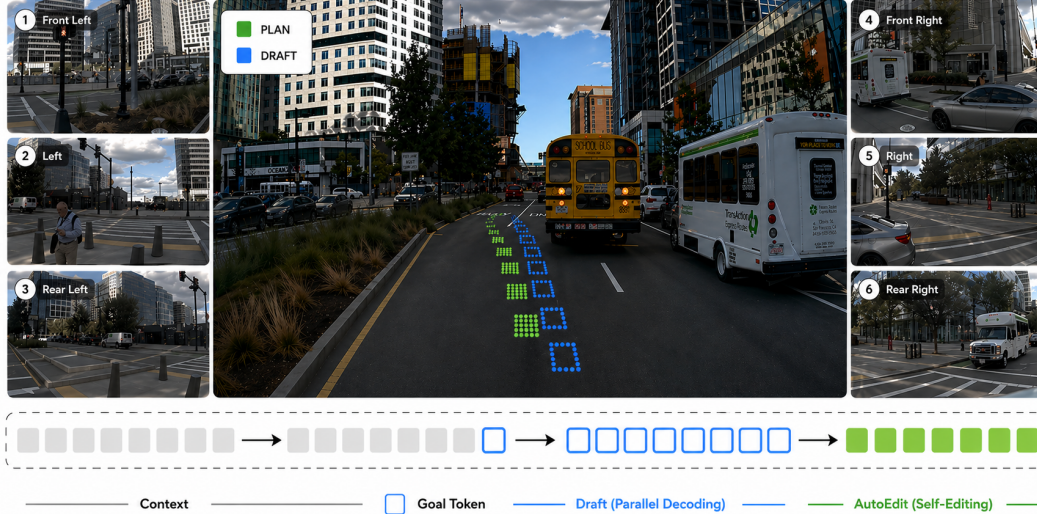


Figure 1: **ReflectDrive-2 plans through a decision-draft-reflect process in a shared discrete token space.** Conditioned on surround-view cameras, navigation, and ego-state, a goal-point posterior commits a single Goal Token, masked discrete diffusion parallel-decodes a trajectory Draft (blue), and AutoEdit rewrites tokens in place to yield the final Plan (green).

discrete diffusion [Austin et al., 2021, Nie et al., 2025, Song et al., 2025, Bie et al., 2026] admits such revision natively: any subset of trajectory tokens can be re-masked and rewritten by the same model, conditioned on the rest, without an auxiliary network or a separate inference mode.

Simply adding a self-editing step on top of a trained drafter, however, yields little. The drafter has no incentive to emit drafts that the editor can improve, and the editor receives no signal indicating which rewrites pay off in closed-loop behavior. Under supervised training alone, the self-editing capability exists in the weights but the two stages are decoupled: the drafter optimizes its own token-level loss, and the editor optimizes a separate correction loss. Neither stage is aware of the other’s effect on the final driving outcome. Reinforcement learning (RL) over the full draft-and-edit rollout closes this gap. When a single terminal reward assigns policy-gradient credit to both drafting and editing transitions, the two phases become coupled. The drafter learns to emit revisable drafts – token distributions whose post-edit trajectory scores higher than the pre-edit one – and the editor learns corrections that move the draft toward the closed-loop reward rather than only reducing token-level uncertainty. Self-correction is no longer a post-hoc add-on; it becomes part of the optimized policy rollout.

We call the resulting system *ReflectDrive-2*, a reflective masked-diffusion VLA planner, and its self-editing mechanism **AutoEdit**. ReflectDrive-2’s inputs are panoramic cameras, route/navigation instruction tokens, and ego state; its outputs are discrete trajectory tokens whose final waypoint tokens anchor a behavior hypothesis, and whose remaining trajectory tokens realize the 4-second plan. Each goal point represents a candidate behavioral hypothesis, such as lane keeping, yielding, overtaking, or changing lanes, and is selected from the predicted goal posterior using top- k sampling with non-maximum suppression. AutoEdit is pretrained against structure-aware perturbations spanning the longitudinal and lateral failure axes above, and then co-trained with the drafter through RL over the joint rollout. Vision and natural-language instructions serve as joint conditioning inputs to a shared backbone that denoises discrete action tokens, and drafting together with AutoEdit constitutes a unified policy loop optimized using a single reward signal.

The reflective structure also shapes the runtime. The inference path (context prefill, goal proposal, multi-batch drafting, AutoEdit) admits a reflection-aware stack with shared-prefix KV cache reuse across the decision-draft-reflect phases, Alternating Step Decode (ASD) that reuses AutoEdit across frames as a temporal refiner, and a fused on-device unmasking kernel. On NAVSIM [Dauner et al., 2024], ReflectDrive-2 reaches 91.0 PDMS camera-only, and 94.8 PDMS under best-of-6 oracle selection; on NVIDIA Thor the stack averages 30.2 ms per frame.

To summarize, our main contributions are as follows:

- **Goal-conditioned masked-diffusion planning.** We propose *ReflectDrive-2*, a driving VLA that plans through a *decision–draft–reflect* process. A goal-point posterior exposes behavior-level hypotheses; masked discrete diffusion drafts editable trajectories for each hypothesis; and AutoEdit rewrites drafts in the same token space. On NAVSIM, ReflectDrive-2 achieves 91.0 PDMS with camera-only input, and 94.8 PDMS under best-of-6 oracle selection.
- **Reward-coupled AutoEdit.** We introduce **AutoEdit**, a self-correction mechanism trained with structure-aware perturbations that match the longitudinal and lateral failure axes of imitation-learned driving. By applying RL over the full draft-and-edit rollout, the reward signal co-adapts drafter and editor, substantially amplifying the effectiveness of inference-time AutoEdit.
- **Efficient reflective decoding.** We co-design a runtime stack that exploits the decision–draft–reflect structure: shared-prefix KV cache, ASD reinterpreted as temporal AutoEdit, and fused CUDA unmasking, achieving 30.2 ms average latency on NVIDIA Thor with near-lossless planning quality.

2 Related Work

2.1 End-to-End and VLA Planning

End-to-end planners map sensors to trajectories without inter-module error propagation [Chitta et al., 2022, Hu et al., 2023, Jiang et al., 2023a, Hu et al., 2022]; SMART [Feng et al., 2024] tokenizes multi-agent trajectories for autoregressive next-token prediction. VLA planners [Li et al., 2025a, Zhou et al., 2025, Li et al., 2025b, Kim et al., 2024, Tian et al., 2024, Sima et al., 2024] inherit language priors but decode token-by-token, so latency scales with trajectory length and any correction requires a second sequential rollout. Continuous diffusion planners [Janner et al., 2022, Chi et al., 2023, Liao et al., 2025, Xing et al., 2025, Zheng et al., 2026] generate in parallel but require ≥ 20 denoising steps, and guided variants [Zhong et al., 2023, Jiang et al., 2023b] compound cost through per-step gradient propagation. ReflectDrive-2 replaces both paradigms with masked discrete diffusion: parallel unmasking reaches a full trajectory in a few rounds, and token-level editing is native rather than a second-stage add-on. These baselines do not naturally couple in-place editing with the same policy rollout and reward signal – the property that our approach builds on.

2.2 Discrete Diffusion and Token-Space Editing

Discrete diffusion provides a natural generative framework for categorical state spaces. D3PM [Austin et al., 2021] extends diffusion modeling to discrete variables, and MaskGIT [Chang et al., 2022] shows that masked-token prediction can support parallel generation through confidence-based unmasking. This line has recently scaled to language modeling: LLaDA [Nie et al., 2025] and Seed Diffusion [Song et al., 2025] train large masked-diffusion language models, while MDLM [Lou et al., 2024a], SEDD [Lou et al., 2024b], Block Diffusion [Arriola et al., 2025], and Fast-dLLM [Wu et al., 2025] improve the formulation or serving efficiency of discrete diffusion models. LLaDA 2.0/2.1 [Bie et al., 2025, 2026] further scale this paradigm and introduce Token-to-Token (T2T) editing, where low-confidence tokens are regenerated during decoding.

The ability to re-mask and regenerate arbitrary token subsets makes discrete diffusion especially suitable for editable planning. However, most existing token-editing mechanisms are either decoding-time heuristics or independently trained refinement stages. LLaDA 2.1 T2T [Bie et al., 2026], for example, revises tokens according to model confidence, but the model is not explicitly trained on the structured errors that arise in downstream control. In contrast, AutoEdit is supervised with trajectory perturbations aligned with common driving failure modes, including longitudinal progress errors and lateral heading deviations. The editor therefore observes the types of failures it is expected to correct during training, rather than relying only on uncertainty estimates at inference time.

Recent work has also explored refinement in embodied or multimodal diffusion models. Drive-Fine [Dang et al., 2026] is the closest prior work, introducing a refinement-augmented masked-diffusion driving VLA. Its refiner, however, is trained and optimized separately from the drafter. ReflectDrive-2 instead treats drafting and editing as a single composed rollout: the terminal driving

reward is assigned to the post-edit trajectory, and policy-gradient credit is applied to token transitions from both stages. This joint credit assignment allows the drafter and editor to co-adapt under the same closed-loop objective. Similarly, “From denoising to refining” [Ji et al., 2025] studies corrective refinement for vision–language diffusion models, but focuses on multimodal understanding rather than closed-loop control and does not couple the refiner to a driving reward. LLaDA-VLA [Wen et al., 2025] applies discrete diffusion to robot control, while ReflectDrive-2 focuses on token-space editing for autonomous driving and optimizes the draft–edit process through a shared rollout reward.

2.3 Reinforcement Learning for Diffusion Policies

DDPO [Black et al., 2024] and DPPO [Ren et al., 2025] apply policy gradients to continuous diffusion by treating denoising as a multi-step MDP, which requires reparameterization in continuous state spaces. For discrete diffusion, d1 [Zhao et al., 2025] uses GRPO-style RL but ignores multi-step structure; d2 [Wang et al., 2025b] recovers it with step-aware gradients and group-relative advantage; SPG [Wang et al., 2025a] derives tighter ELBO/EUBO bounds. In driving, HDP [Zheng et al., 2026] and DriveFine [Dang et al., 2026] adopt RL post-training on diffusion planners. These methods each optimize a *single-pass* rollout: drafting alone, or refining alone. ReflectDrive-2’s RL objective is applied to a *composed* rollout, draft $\rightarrow$ AutoEdit, so the terminal reward credits both stages jointly. Simply increasing the number of diffusion steps does not expose a semantically distinct edit operator to receive reward credit; our composed rollout contains a reflection phase that shares the reward with drafting. Section 4.5 formalizes the distinction and Table 3 isolates the substantial amplification of the editor’s gain that results.

3 Preliminaries

3.1 Problem Setting

At time step t , the ego vehicle receives an observation $\mathbf{o}_t = (\mathbf{v}_t, \ell_t, \mathbf{s}_t)$ with three channels: panoramic visual tokens $\mathbf{v}_t$ from left-front, front, and right-front cameras over two temporal frames; a *navigational instruction* channel ℓ_t carrying route-level commands and maneuver hints (keep lane, turn left at intersection, proceed straight) as linguistic tokens consumed by the same backbone that models action tokens; and an ego-state channel $\mathbf{s}_t$ with kinematic tokens (velocity, acceleration, yaw rate). The instruction channel is the “L” of our VLA: it conditions drafting on intent, not just on scene. The objective is to generate a future trajectory $\tau = \{(x_k, y_k)\}_{k=1}^K$ that is safe, comfortable, rule-compliant, and consistent with ℓ_t . Heading is derived from consecutive waypoints when required by downstream metrics.

3.2 Masked Discrete Diffusion

Forward and reverse process. We represent the future ego trajectory as a sequence of Bird’s-Eye-View (BEV) coordinate tokens, denoted by $\mathbf{x}_0$. Following masked discrete diffusion [Austin et al., 2021, Nie et al., 2025], the forward process corrupts $\mathbf{x}_0$ by independently replacing each token with [MASK] at probability $t \in [0, 1]$, yielding a partially masked sequence $\mathbf{x}_t$. A bidirectional Transformer p_θ reverses this process by predicting the original tokens from $\mathbf{x}_t$ conditioned on multimodal context $\mathbf{c} = (\mathbf{v}_t, \ell_t, \mathbf{s}_t)$. Prior masked-diffusion language models typically optimize a $1/t$ -weighted cross-entropy on masked positions only [Nie et al., 2025]; we supervise *all* positions:

$$\mathcal{L}_{\text{DLM}}(\theta) = -\mathbb{E}_{\mathbf{x}_0, t} \left[\frac{1}{L} \sum_{i=1}^L \log p_\theta(x_0^i \mid \mathbf{x}_t, \mathbf{c}) \right]. \quad (1)$$

Empirically the all-position objective yields more stable optimization and coherent drafts. At inference time, generation begins from a fully masked sequence and proceeds through a small number of parallel denoising steps.

Selective re-generation. Masked diffusion admits arbitrary in-place rewriting: for any edit mask $\mathbf{e} \in \{0, 1\}^L$, the partial sequence $\tilde{\mathbf{x}} = \mathbf{e} \odot [\text{MASK}] + (1 - \mathbf{e}) \odot \mathbf{x}_0$ is denoised from effective time $t^* \approx |\mathbf{e}|/L$. LLaDA 2.1 [Bie et al., 2026] extends this idea through Token-to-Token (T2T) editing, which also revises low-confidence tokens at decoding time. Our AutoEdit framework inherits this

interface but shifts the editor from decoding-time heuristic to trained operator (Section 4.3) and couples it to the drafter through a shared RL reward (Section 4.5).

3.3 KV Caching for Efficient Inference

Standard masked diffusion uses bidirectional attention, so vanilla KV caching fails: KV entries must be recomputed at every denoising step because masked tokens change [Nie et al., 2025]. Block Diffusion [Arriola et al., 2025] partitions the sequence into blocks, running diffusion within a block and generating blocks autoregressively for cache reuse on completed blocks. LLaDA 2.1 [Bie et al., 2026] generalizes to block-wise causal attention, and LLaDA 2.0 [Bie et al., 2025] adds serving-level optimizations such as variable-length batching and prefix caching in its dInfer engine. We adopt causal attention over the scene-context prompt and block-wise attention over trajectory tokens, which permits KV reuse for the prompt while preserving bidirectional diffusion within the trajectory block (Section 5).

3.4 Reinforcement Learning Fine-Tuning

Supervised training imitates the data distribution but does not optimize driving objectives directly. We cast trajectory generation as a Markov decision process and fine-tune with reinforcement learning so the policy is aligned with a closed-loop reward. Following Wang et al. [2025b], the objective is $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta(\cdot|o)}[R(\tau)]$, optimized with group-relative advantage over G sampled trajectories and a discrete-diffusion policy gradient:

$$\mathcal{L}(\theta) = -\mathbb{E}_{\tau_1:G \sim \pi_{\theta_{\text{old}}}(\cdot|o)} \left[\frac{1}{G} \sum_{g=1}^G \frac{1}{L} \sum_{s=1}^S \sum_{p=1}^L \mathbf{1}_{\{x_{g,p}^{s+1} \neq x_{g,p}^s\}} \cdot \min(r_{g,p}^s A_g, \text{clip}(r_{g,p}^s, 1 - \epsilon, 1 + \epsilon) A_g) \right] + \lambda_{\text{KL}} D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}), \quad (2)$$

where S is the total number of generation steps, $A_g = R(\tau_g) - \frac{1}{G} \sum_j R(\tau_j)$, and $r_{g,p}^s = \pi_\theta(x_{g,p}^s | \mathbf{x}_{g,s+1}, o) / \pi_{\theta_{\text{old}}}(x_{g,p}^s | \mathbf{x}_{g,s+1}, o)$. The indicator $\mathbf{1}_{\{x_{g,p}^{s+1} \neq x_{g,p}^s\}}$ restricts credit to tokens that are actually updated at step s . In Section 4.5 we instantiate $S = S_{\text{draft}} + S_{\text{edit}}$, so the same reward credits token transitions from drafting and AutoEdit jointly – the methodological centerpiece of this paper.

4 Method

4.1 ReflectDrive-2 Overview

ReflectDrive-2 formulates autonomous driving planning as goal proposal, masked trajectory drafting, and token-space trajectory correction within a unified discrete representation. Given multimodal driving context $\mathbf{c} = (\mathbf{v}_t, \ell_t, \mathbf{s}_t)$, where $\mathbf{v}_t$ denotes visual tokens, ℓ_t denotes route-instruction tokens, and $\mathbf{s}_t$ denotes ego-state tokens, the model first predicts a set of goal-point hypotheses. Each goal is then used to condition a masked discrete-diffusion decoder that generates a trajectory in parallel over a small number of denoising rounds. After the initial draft is produced, AutoEdit reuses the same conditional token model to update selected trajectory tokens. The planner therefore performs generation and correction in the same action-token space, without introducing a separate refinement network.

The method has three coupled components. First, a goal-point posterior provides a compact decision layer over behavior-level hypotheses, such as different turning lines, yielding behavior, or passing around another agent. Second, goal-conditioned masked diffusion realizes each selected hypothesis as a full trajectory by filling discrete BEV coordinate tokens. Third, AutoEdit performs token-space correction by selectively rewriting parts of the drafted trajectory. The supervised stage trains both masked trajectory generation and structure-aware correction: standard random masking teaches the model to draft trajectories, while perturbation-based correction teaches it to recover clean trajectories from longitudinal and lateral planning errors. A constraint-aware field loss further regularizes the spatial distribution of predicted tokens against drivable-area geometry.

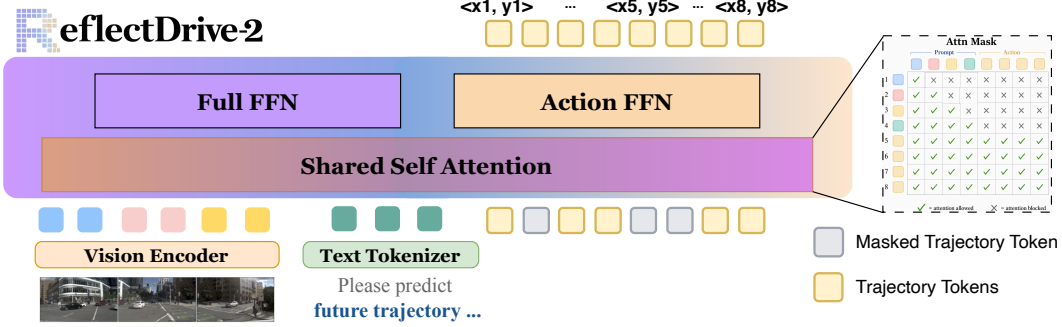


Figure 2: **ReflectDrive-2 architecture.** Vision, language, ego-state, and trajectory tokens share a single self-attention backbone, with a Full FFN over prompt tokens and a compact Action FFN over the action-token block. The attention pattern (right) is causal over the prompt, enabling shared-prefix KV reuse and block-wise bidirectional over action tokens.

The reinforcement-learning stage optimizes the complete draft-and-edit rollout rather than the drafting stage alone. For each sampled candidate, the terminal driving reward is assigned to the final post-edit trajectory, and policy-gradient credit is applied to token transitions from both the drafting and AutoEdit phases. This coupling is central to ReflectDrive-2: AutoEdit is not treated as a post-processing heuristic, but as part of the policy rollout that is optimized under the same closed-loop objective as the drafter. The complete inference path can be summarized as

$$\mathbf{c} \rightarrow \{g_m\}_{m=1}^{N_g} \rightarrow \{\mathbf{x}_m^{(0)}\}_{m=1}^{N_g} \rightarrow \{\mathbf{x}_m^{(K)}\}_{m=1}^{N_g}, \quad (3)$$

where g_m is a sampled goal point, $\mathbf{x}_m^{(0)}$ is the drafted trajectory conditioned on g_m , and $\mathbf{x}_m^{(K)}$ is the trajectory after K AutoEdit rounds. Vision tokens, route-instruction tokens, ego-state tokens, goal tokens, and trajectory tokens are processed by the same backbone, while diffusion denoising is applied to the action-token block. This shared token substrate allows trajectory drafting and editing to be trained and optimized as one action-generation process.

4.2 Goal-Conditioned Masked Trajectory Diffusion

Multimodal context encoding. Two temporally adjacent panoramic frames from the left-front, front, and right-front cameras are encoded by a ViT visual backbone and projected into the diffusion Transformer’s token space. The resulting visual tokens are concatenated with route-instruction tokens ℓ_t and ego-state tokens s_t , and the concatenated sequence is processed by the shared backbone. Each Transformer block additionally contains an action-specific FFN and an action head, which specialize the model for trajectory-token prediction while retaining the shared backbone for scene and context modeling.

Goal-point prediction. Rather than committing to a unimodal endpoint prediction, ReflectDrive-2 predicts a goal-point posterior over discrete BEV coordinates. A goal point is represented as a discrete (x, y) token pair and serves as a behavior-level hypothesis for the future plan. During training, the goal head is supervised by the expert endpoint. During inference, we sample candidate goals using top- k sampling followed by non-maximum suppression (NMS) in BEV space. NMS removes duplicate endpoints while preserving spatially distinct alternatives, so different surviving goals can correspond to different maneuvers, such as lane keeping versus yielding, pass-left versus pass-right, or different feasible lines through a turn. Each selected goal conditions a separate masked-diffusion drafting branch.

Masked trajectory drafting. We represent the future ego trajectory over the benchmark planning horizon with 8 waypoints. Each waypoint is discretized into one longitudinal and one lateral coordinate token, yielding a length- $L = 16$ trajectory sequence

$$\mathbf{x}_0 = [x_1, y_1, \dots, x_8, y_8], \quad (4)$$

where the final coordinate pair (x_8, y_8) corresponds to the selected goal. During supervised training, random positions are replaced by [MASK] and the model is trained with the all-position masked-diffusion objective in Eq. (1). At inference time, the selected goal tokens are fixed, the remaining

trajectory tokens are initialized as [MASK], and the model fills masked positions over a small number of parallel denoising rounds. At each round, the most confident predictions are committed. The generation cost is therefore determined by the number of denoising rounds rather than the number of trajectory tokens, and the same masked-token interface later enables selective trajectory rewriting.

4.3 AutoEdit Trajectory Correction

AutoEdit is a token-to-token trajectory editor operating in the same discrete action space as the masked-diffusion drafter. Unlike masked trajectory drafting, AutoEdit does not convert selected trajectory tokens back to [MASK]. Instead, it takes the current concrete trajectory-token sequence as input, predicts replacement tokens at trajectory positions, and commits only the selected replacements. Thus, AutoEdit performs direct token-to-token rewriting rather than re-masking and re-denoising.

Structure-aware perturbations. Given a clean waypoint sequence $\mathbf{z}_0 = \{\mathbf{z}_i\}_{i=1}^N$, we synthesize a perturbed trajectory $\tilde{\mathbf{z}}_0 = \mathcal{T}(\mathbf{z}_0)$ before tokenization. The perturbation operator $\mathcal{T}$ targets two common planning-error families: longitudinal progress errors and lateral heading deviations.

Longitudinal progress perturbation. We rescale progress along the trajectory arc length:

$$\tilde{\mathbf{z}}_i = \text{Interp}(\mathbf{z}_0, \beta d_i), \quad \beta \sim \mathcal{U}(\beta_{\min}, \beta_{\max}), \quad (5)$$

where d_i is the arc length at waypoint $\mathbf{z}_i$. Values $\beta < 1$ produce conservative under-progress, while $\beta > 1$ produces overshoot or insufficient deceleration.

Lateral heading perturbation. We rotate the trajectory in the ego frame:

$$\tilde{\mathbf{z}}_i = \mathbf{R}(\alpha)\mathbf{z}_i, \quad \mathbf{R}(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix}, \quad \alpha \sim \mathcal{U}(-\alpha_{\max}, \alpha_{\max}). \quad (6)$$

This produces coherent lateral deviation while preserving trajectory smoothness.

After tokenizing the perturbed trajectory into $\tilde{\mathbf{x}}_0$, AutoEdit is trained to map the perturbed token sequence directly back to the clean token sequence:

$$q_\theta(\cdot \mid \tilde{\mathbf{x}}_0, \mathbf{c}) = \text{softmax}(h_\theta(\tilde{\mathbf{x}}_0, \mathbf{c})), \quad (7)$$

where h_θ is the shared conditional token model used by the planner. The structure-aware AutoEdit loss is

$$\mathcal{L}_{\text{SAP}}(\theta) = -\mathbb{E}_{\mathbf{x}_0, \mathcal{T}} \left[\frac{1}{L} \sum_{i=1}^L \log q_\theta(x_0^i \mid \tilde{\mathbf{x}}_0, \mathbf{c}) \right]. \quad (8)$$

This objective teaches the model to directly translate perturbed trajectory tokens into clean trajectory tokens, rather than to recover clean tokens from a newly masked sequence.

Inference-time AutoEdit. At test time, AutoEdit starts from a drafted trajectory $\mathbf{x}^{(0)}$ and performs K token-to-token editing rounds. At round k , the model predicts a replacement-token sequence from the current trajectory tokens:

$$\hat{\mathbf{x}}^{(k+1)} = \text{T2TEdit}_\theta(\mathbf{x}^{(k)}, \mathbf{c}). \quad (9)$$

We then compute a commit mask $\mathbf{m}^{(k)} \in \{0, 1\}^L$, where $\mathbf{m}_i^{(k)} = 1$ means that the replacement token at position i is committed. In the default setting, $\mathbf{m}^{(k)}$ selects low-confidence non-goal trajectory tokens, while the goal tokens remain fixed as the behavior anchor. The update is

$$\mathbf{x}^{(k+1)} = \mathbf{m}^{(k)} \odot \hat{\mathbf{x}}^{(k+1)} + (1 - \mathbf{m}^{(k)}) \odot \mathbf{x}^{(k)}. \quad (10)$$

Importantly, $\mathbf{m}^{(k)}$ is a commit mask, not a re-masking mask. the input to AutoEdit remains the concrete token sequence $\mathbf{x}^{(k)}$, and no selected token is converted back to [MASK]. AutoEdit is therefore a direct token-to-token trajectory editor implemented by the same conditional token model as the drafter, without an auxiliary refinement network or a hand-designed smoothing module.

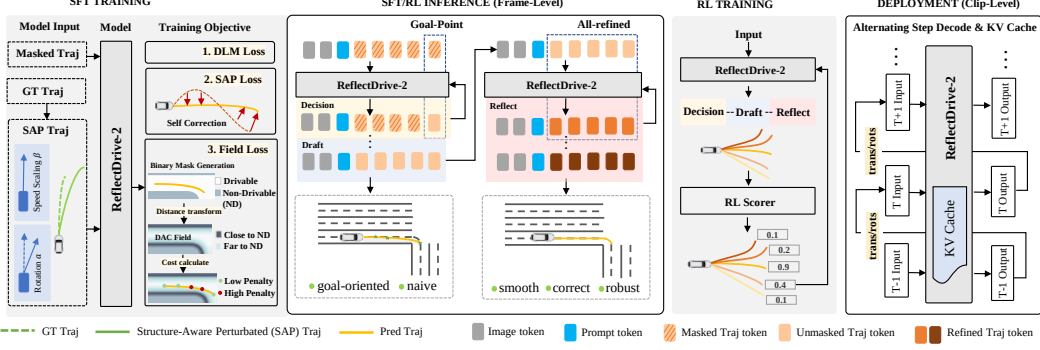


Figure 3: **Training, inference, and deployment of ReflectDrive-2.** (a) SFT with three objectives: masked-token loss $\mathcal{L}_{\text{DLM}}$, structure-aware perturbation loss $\mathcal{L}_{\text{SAP}}$ over longitudinal scaling β and lateral rotation α , and drivable-area field loss $\mathcal{L}_{\text{field}}$. (b) Frame-level *decision-draft-reflect* inference: commit a goal token, parallel-decode the draft, then rewrite tokens in place via AutoEdit. (c) RL fine-tuning scores full draft-and-edit rollouts with a closed-loop reward, propagating group-relative credit through both phases. (d) Clip-level deployment alternates full-step and lite-step frames with shared-prefix KV reuse; rigid-body transforms carry the previous plan into the current ego frame.

4.4 Constraint-Aware Supervised Objectives

Drivable-area field loss. The masked-diffusion loss $\mathcal{L}_{\text{DLM}}$ and the AutoEdit correction loss $\mathcal{L}_{\text{SAP}}$ optimize token-level prediction, but they do not explicitly encode drivable-area geometry. We add a field-based spatial penalty over the waypoint distribution induced by the coordinate-token logits. Let $p_{xy}^{(t)} \in \mathbb{R}^{H \times W}$ denote the spatial distribution at waypoint t , obtained from the marginal coordinate distributions as

$$p_{xy}^{(t)}[i, j] = p_x^{(t)}[i] p_y^{(t)}[j]. \quad (11)$$

Given a BEV cost field $\mathcal{C} \in \mathbb{R}_{\geq 0}^{H \times W}$, we penalize probability mass assigned to high-cost cells using a field-weighted log barrier:

$$\mathcal{L}_{\text{field}} = \sum_{t=1}^T \sum_{i,j} -\log(1 - p_{xy}^{(t)}[i, j]) \mathcal{C}[i, j]. \quad (12)$$

The logarithmic factor gives larger gradients when the model assigns high confidence to high-cost regions.

In our implementation, $\mathcal{C}$ is instantiated as a drivable-area compliance field. Let $\mathbf{b} \in \{0, 1\}^{H \times W}$ be the drivable-area indicator. The outside distance is defined as

$$d_{\text{out}}[i, j] = \begin{cases} 0, & \mathbf{b}[i, j] = 1, \\ r_{\text{DAC}} \min_{(u,v): \mathbf{b}[u,v]=1} \sqrt{(i-u)^2 + (j-v)^2}, & \mathbf{b}[i, j] = 0. \end{cases} \quad (13)$$

The DAC cost field is then

$$\mathcal{C}_{\text{DAC}}[i, j] = \max(0, d_{\text{out}}[i, j] - \epsilon_{\text{safe}}), \quad (14)$$

where ϵ_{safe} defines a tolerance band near the drivable-area boundary. We isolate the contribution of the field loss in Table 6.

Total supervised objective. The full supervised objective combines masked trajectory generation, structure-aware correction, and drivable-area regularization:

$$\mathcal{L}_{\text{sup}} = \mathcal{L}_{\text{DLM}} + \lambda_{\text{SAP}} \mathcal{L}_{\text{SAP}} + \lambda_{\text{field}} \mathcal{L}_{\text{field}}. \quad (15)$$

4.5 Reinforcement Learning over Draft-and-Edit Rollouts

Supervised training teaches the model to imitate expert trajectories and recover from synthetic perturbations, but it does not directly optimize closed-loop driving metrics. We therefore fine-tune

ReflectDrive-2 with reinforcement learning over the composed draft-and-edit rollout. The key distinction from a longer single-pass diffusion rollout is that the generation process is explicitly divided into a drafting phase and an AutoEdit phase. The terminal reward is assigned to the final post-edit trajectory, and the policy-gradient objective credits token transitions from both phases.

For each scene, we sample N_g goal points by top- k sampling with NMS and draw I drafts per goal, giving $G = N_g I$ candidate rollouts. For candidate g , the token-transition sequence is

$$\rho_g = (\mathbf{x}_g^0, \mathbf{x}_g^1, \dots, \mathbf{x}_g^{S_{\text{draft}}}, \mathbf{x}_g^{S_{\text{draft}}+1}, \dots, \mathbf{x}_g^{S_{\text{draft}}+S_{\text{edit}}}), \quad (16)$$

where the first S_{draft} transitions correspond to masked trajectory drafting and the next S_{edit} transitions correspond to AutoEdit. The final trajectory is

$$\tau_g = \text{Detok}(\mathbf{x}_g^{S_{\text{draft}}+S_{\text{edit}}}). \quad (17)$$

We use the closed-loop planning score as the terminal reward $R(\tau_g)$ and compute a group-relative advantage

$$A_g = R(\tau_g) - \frac{1}{G} \sum_{j=1}^G R(\tau_j). \quad (18)$$

The discrete-diffusion policy-gradient objective in Eq. (2) is applied over all token transitions in ρ_g . Equivalently, the token-transition indicator

$$\delta_{g,p}^s = \mathbf{1}_{\{\mathbf{x}_{g,p}^{s+1} \neq \mathbf{x}_{g,p}^s\}}, \quad s = 0, \dots, S_{\text{draft}} + S_{\text{edit}} - 1, \quad (19)$$

covers both unmasking during drafting and rewriting during AutoEdit. The same terminal reward therefore optimizes goal-conditioned drafting and AutoEdit under one rollout objective. Because only the post-edit trajectory receives reward, the drafting phase is optimized for trajectories that can be improved by the subsequent correction phase, while AutoEdit is optimized for corrections that improve the closed-loop score rather than only reducing token-level uncertainty.

5 Efficient Inference for Reflective Masked Planning

We treat deployment as an optimization chain rather than as independent serving tricks. The optimization sequence is summarized in Table 1. Each row stacks one optimization on top of the previous and reports the resulting end-to-end planner latency on NVIDIA Thor. The final stack runs a full decision-draft-reflect pass on full-step frames and a lightweight temporal AutoEdit pass on lite-step frames, yielding an average latency of 30.2 ms per frame.

Shared-prefix KV reuse. Goal-point proposal, trajectory drafting, and AutoEdit all condition on the same visual, route-instruction, and ego-state prefix. Instead of recomputing this prefix for each phase, we keep a shared prefix cache and switch between `Single` and `Batch` cache states according to the current serving phase. This reduces the decode latency from 56.8 ms to 42.2 ms.

Mutable action-cache rewinding and merged rewrite. The action-token block is mutable: masked drafting changes [MASK] tokens into concrete trajectory tokens, while AutoEdit directly replaces concrete trajectory tokens with revised concrete tokens. In both cases, KV entries associated with the previous action-token state become stale. After each update, the cache pointer is rewound to the shared-prefix boundary, and only the mutable action block is recomputed. At multi-block boundaries, we further merge the required cache rewrite with the first token-update step of the next block, reducing the boundary latency by approximately 3 ms. Similarly, by fusing the prefill and keypoint inference into a single pass, the prefill-plus-keypoint stage latency drops from 14.4 ms to 13.8 ms.

Action-expert FFN. Trajectory-token decoding uses a constrained action vocabulary and a short fixed-length token block. We therefore replace the full FFN in the action branch with a compact action-expert FFN, reducing the hidden dimension from 4096 to 1024. This lowers the decode latency from 42.2 ms to 29.6 ms. And we measure the action-expert FFN via trajectory-level metrics summarized in Table 2 to validate its feasibility. Although minSADE slightly increases, the compact branch improves meanSADE and the selected path-level error metrics.

Table 1: **Inference optimization chain on NVIDIA Thor.** All latencies in ms. Each row stacks one optimization on top of the previous; “—” denotes unchanged from the row above. On ASD lite-step frames, keypoint proposal(Decision) is skipped and only two decode steps are run.

Optimization	Prefill + Decision	Decode	Total
Baseline	14.4	56.8	71.2
+ Merged infer	13.8	—	70.6
+ Shared-prefix KV reuse	—	42.2	56.0
+ Action-expert FFN	—	29.6	43.4
+ Fused CUDA unmasking	—	27.3	41.1 (full-step)
+ ASD (lite-step)	9.8	9.5	19.3 (lite-step)

Table 2: **Quality gates for deployment optimizations.** Δ denotes optimized – baseline. **Green** marks the desired direction; **red** marks regressions. For driving scores ($\uparrow$) higher is better; for trajectory errors ($\downarrow$) lower is better.

(a) ASD temporal AutoEdit on the in-house deployment benchmark ($\uparrow$)							
Setting	NC	DAC	TTC	Comf.	EP	Overall	
Full pipeline (baseline)	79.51	76.99	96.27	88.52	75.13	83.70	
ASD	79.19	77.21	96.20	87.88	75.01	83.51	
Δ	−0.31	+0.22	−0.07	−0.63	−0.12	−0.20	

(b) Action-expert FFN trajectory quality ($\downarrow$)								
Setting	Trajectory-level		path-meanFDE			path-minADE		
	minSADE	meanSADE	@20	@40	@80	@20	@40	@80
Original FFN (baseline)	0.858	1.710	0.167	0.427	0.991	0.088	0.210	0.462
Action-expert FFN	0.914	1.539	0.145	0.346	0.910	0.084	0.177	0.406
Δ	+0.056	−0.171	−0.022	−0.081	−0.081	−0.004	−0.033	−0.056

Fused on-device token update. Both masked drafting and token-to-token AutoEdit require confidence ranking, token selection, and state update. In masked drafting, the update commits predicted tokens in place of [MASK]; in AutoEdit, the update overwrites selected concrete tokens with replacement tokens. A CPU implementation introduces device synchronization at every step. We fuse token selection, ranking, and token-state update into an on-device CUDA kernel, reducing the per-step update latency from 29.6 ms to 27.3 ms.

Alternating Step Decode as temporal token-to-token AutoEdit. In streaming driving, adjacent frames share scene context and future plans. ReflectDrive-2 therefore alternates between full-step and lite-step frames. A full-step frame runs the complete decision–draft–reflect pipeline. A lite-step frame transforms the previous plan into the current ego frame and applies a short token-to-token AutoEdit update instead of rebuilding the trajectory from scratch:

$$\tilde{\tau}^{(t)} = \mathcal{T}_{t-1 \rightarrow t} \left(\text{Shift} \left(\tilde{\tau}^{(t-1)} \right) \right), \quad \mathbf{x}^{(t,0)} = \text{Tok} \left(\tilde{\tau}^{(t)} \right), \quad \hat{\mathbf{x}}^{(t)} = \text{T2TEdit}_{\theta}^{S'} \left(\mathbf{x}^{(t,0)}, \mathbf{c}^{(t)} \right). \quad (20)$$

Here $\text{Shift}(\cdot)$ removes the elapsed portion of the previous plan, and $\mathcal{T}_{t-1 \rightarrow t}$ transforms the remaining waypoints from the previous ego frame to the current ego frame. We use $S = 1 + 3 + 3$ decision–draft–reflect steps for full-step frames and $S' = 1 + 1$ draft–reflect steps for lite-step frames, reducing the decode latency from 27.3 ms to 9.5 ms. And we also evaluate ASD against running the full pipeline on every frame in Table 2. Replacing alternating full frames with temporal token-to-token AutoEdit changes the in-house overall score by only -0.20 , while drivable area compliance slightly improves. Overall, the resulting planner runs at 30.2 ms average latency on NVIDIA Thor, with full-step frames at 41.1 ms and lite-step frames at 19.3 ms. Thus, the same token-to-token AutoEdit operator serves both as an intra-frame trajectory corrector and as an inter-frame temporal refiner.

Table 3: **Effect of inference-time AutoEdit across training regimes.** After supervised training, inference-time AutoEdit contributes at most +0.3 PDMS. After RL over the full draft-and-edit rollout, the same inference-time AutoEdit contributes +1.9 PDMS.

Training setting	w/o AutoEdit	w/ AutoEdit	Δ PDMS
DLM	84.8	85.0	+0.2
DLM + DACF	87.2	87.3	+0.1
DLM + DACF + AutoEdit training	87.7	88.0	+0.3
DLM + DACF + AutoEdit training + RL	89.1	91.0	+1.9

6 Experiments

6.1 Experimental Setup

Dataset and metrics. We evaluate ReflectDrive-2 on NAVSIM [Dauner et al., 2024], a closed-loop planning benchmark built on nuPlan [Caesar et al., 2021]. The task is to predict a 4-second ego trajectory at 2 Hz. We train on navtrain (1,192 scenes) and evaluate on navtest (136 scenes). The metric is Predictive Driver Model Score (PDMS), aggregating no at-fault collision (NC), drivable-area compliance (DAC), time to collision (TTC), comfort, and ego progress (EP).

Implementation. A 0.7B masked-diffusion language backbone and a 0.1B ViT visual encoder, both initialized from proprietary pretrained weights, are fully fine-tuned on NAVSIM. The input is two temporal frames from the left-front/front/right-front cameras plus navigation instruction and ego-state tokens; the output is 8 waypoints represented as 16 discrete coordinate tokens. We supervise-fine-tune first, then reinforcement-fine-tune with PDMS as reward.

Baselines. End-to-end planners: UniAD [Hu et al., 2023], TransFuser [Chitta et al., 2022], HydraMDP [Li et al., 2024], DiffusionDrive [Liao et al., 2025], GoalFlow [Xing et al., 2025]. VLA planners: AutoVLA [Zhou et al., 2025], DriveVLA-W0 [Li et al., 2025a], ReCogDrive [Li et al., 2025b]. For standard evaluation all methods emit one trajectory; for best-of- N evaluation, ReflectDrive-2 samples multiple goal points and keeps the trajectory with the highest closed-loop score (oracle selection). In standard evaluation, we use the highest-confidence goal after NMS and output one trajectory. Best-of-6 evaluation reports the oracle-best over six candidate trajectories sampled from six goal-point proposals; during RFT the same group size of six is composed as three goal points with two drafts each.

6.2 Effect of RL on Inference-Time AutoEdit

Table 3 isolates the main interaction behind our final result. Before RL, inference-time AutoEdit delivers at most +0.3 PDMS, regardless of whether AutoEdit was trained with structure-aware perturbation – the editor is learned, but its closed-loop contribution remains modest. After RL over the full draft-and-edit rollout, the same inference-time AutoEdit contributes +1.9 PDMS, a substantial increase relative to the supervised AutoEdit gain. The mechanism is the interaction described in Section 4.5: with a shared terminal reward, the drafter learns to emit revisable drafts (token distributions whose post-edit score exceeds their pre-edit score), and AutoEdit learns corrections that move the draft toward reward rather than only reducing token-level uncertainty. This interaction requires a composed draft-and-edit rollout and does not arise in single-pass RL formulations (Section 2.3). The 89.1 $\rightarrow$ 91.0 improvement that distinguishes ReflectDrive-2 from the strongest camera-only VLA on NAVSIM (Table 4) comes from this interaction.

6.3 Closed-Loop Driving Performance

Table 4 presents standard single-trajectory results as downstream evidence that the RL $\times$ AutoEdit coupling works end-to-end. ReflectDrive-2 reaches 91.0 PDMS with camera-only input, above ReCogDrive (90.8, camera-only) and GoalFlow (90.3, camera and LiDAR). The largest gain is in ego progress (EP = 89.4, highest among listed methods), while DAC remains high at 98.1 and comfort is saturated at 100.0. The result suggests a favorable progress–constraint trade-off: EP improves substantially while DAC and comfort remain high, although NC and TTC are not the best among the

Table 4: **Closed-loop planning results on NAVSIM.** All methods evaluated under standard single-trajectory setting. “C & L” denotes camera and LiDAR.

Method	Input	NC↑	DAC↑	TTC↑	Comf.↑	EP↑	PDMS↑
<i>End-to-End Methods</i>							
UniAD [Hu et al., 2023]	Cam	97.8	91.9	92.9	100.0	78.8	83.4
TransFuser [Chitta et al., 2022]	C & L	97.7	92.8	92.8	100.0	79.2	84.0
Hydra-MDP [Li et al., 2024]	C & L	98.3	96.0	94.6	100.0	78.7	86.5
DiffusionDrive [Liao et al., 2025]	C & L	98.2	96.2	94.7	100.0	82.2	88.1
GoalFlow [Xing et al., 2025]	C & L	98.4	98.3	94.6	100.0	85.0	90.3
<i>Camera-Only VLA Planners</i>							
AutoVLA [Zhou et al., 2025]	Cam	98.4	95.6	98.0	99.9	81.9	89.1
DriveVLA-W0 [Li et al., 2025a]	Cam	98.7	99.1	95.3	99.3	83.3	90.2
ReCogDrive [Li et al., 2025b]	Cam	97.9	97.3	94.9	100.0	87.3	90.8
ReflectDrive-2 (Ours)	Cam	97.3	98.1	92.5	100.0	89.4	91.0

Table 5: **Best-of- N evaluation on NAVSIM.** Best-of-6 uses oracle PDMS selection and is reported to measure the quality of the goal-point posterior, not as a standard benchmark result.

Setting	NC↑	DAC↑	TTC↑	Comf.↑	EP↑	PDMS↑
ReflectDrive-2 (single)	97.3	98.1	92.5	100.0	89.4	91.0
ReflectDrive-2 (best-of-6, oracle)	98.5	99.2	95.5	99.8	93.8	94.8
Human (NAVSIM reference)	100.0	100.0	100.0	99.9	87.5	94.8

listed baselines. Camera-only is the harder setting, and the comparison of interest is against the other camera-only VLA peers (AutoVLA / DriveVLA-W0 / ReCogDrive), over which ReflectDrive-2’s +0.2 to +1.9 PDMS advantage is driven by the rollout-level RL interaction isolated in Table 3.

Table 5 shows that best-of-6 ReflectDrive-2 reaches 94.8 PDMS, reaching the NAVSIM human reference under oracle selection. The gap between single and best-of-6 measures the quality of the goal-point posterior: 3.8 PDMS of headroom is recovered by selecting a different behavior hypothesis per scene, evidence that the goal head exposes a genuinely multi-modal action posterior rather than noisy replicas of the same endpoint.

6.4 Decision Diversity and Reflection

Goal points. Figure 4 visualizes multi-goal inference. In turning scenes (top row), different goal points realize different lines through the curve and some candidates respect the drivable boundary better than others. In interaction scenes (bottom row), the model produces longitudinally and laterally distinct behaviors around nearby agents (keep lane / change lane / adjust speed). Goal points are not sampling noise; they are distinct behavior hypotheses that the downstream decoder realizes.

Reflection with AutoEdit. Figure 5 shows AutoEdit at inference. Semi-transparent curves are initial drafts; solid curves are post-AutoEdit. In the top row, AutoEdit pulls trajectories back into the drivable area; in the bottom row, it adjusts the plan around nearby agents. The revisions are structured rewrites in the same token space used for generation, not cosmetic smoothing.

6.5 Ablation Studies

Training components. Table 6 isolates the effect of each training component without inference-time AutoEdit. Field loss (DACF) contributes +2.4 PDMS (84.8 $\rightarrow$ 87.2) mainly through DAC. AutoEdit supervised training adds +0.5 more. RL over the full rollout brings EP from 82.2 to 89.3 and PDMS to 89.1; combined with inference-time AutoEdit, the final score is 91.0 (Table 3).

Inference budget. Figure 6 sweeps generation steps and AutoEdit steps: performance improves and then plateaus around 3–5 steps, consistent with masked diffusion – a small number of rounds forms a coherent trajectory, while excess rewriting disturbs a good draft.

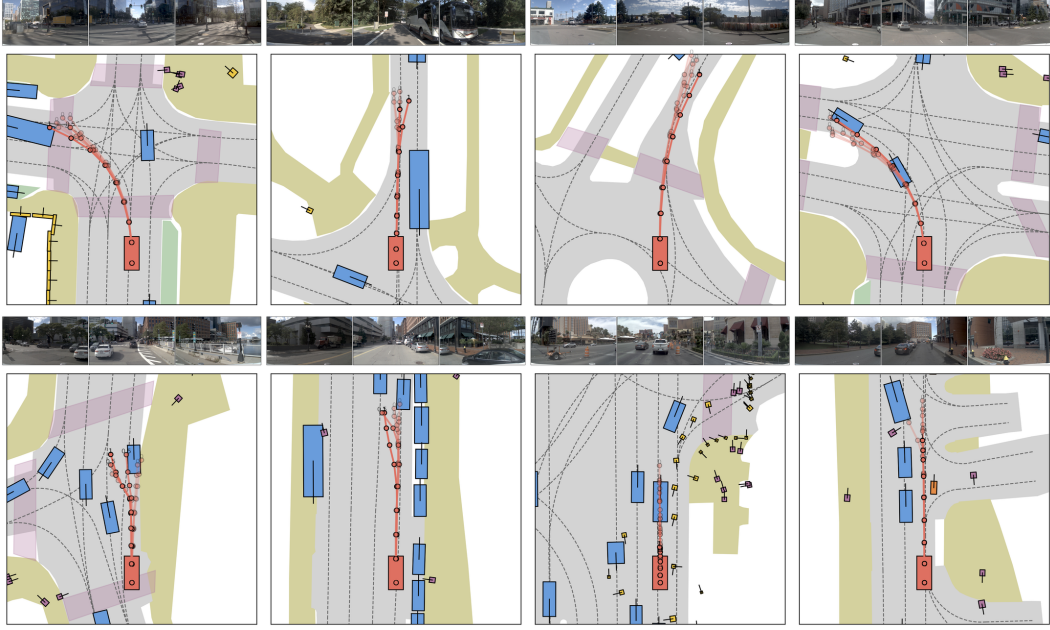


Figure 4: **Decision diversity from goal points.** Each goal anchors a distinct behavior hypothesis. Trajectory opacity encodes PDMS.

Table 6: **Effect of training components.** All rows are evaluated without inference-time AutoEdit to isolate the training signal.

Training objective	NC $\uparrow$	DAC $\uparrow$	TTC $\uparrow$	Comf. $\uparrow$	EP $\uparrow$	PDMS $\uparrow$
DLM	97.5	93.9	92.8	99.5	79.5	84.8
DLM + DACF	97.4	97.0	93.1	99.9	81.4	87.2
DLM + DACF + AutoEdit training	97.8	96.7	93.6	99.9	82.2	87.7
DLM + DACF + AutoEdit training + RL	96.3	97.9	88.9	99.7	89.3	89.1

Goal-proposal parameters. Figure 7 varies the number of goal proposals and the NMS threshold in best-of- N . More proposals expose more behavior hypotheses; an NMS threshold near 1.2 m is optimal – smaller keeps duplicates, larger removes genuine alternatives.

7 Conclusion

We presented *ReflectDrive-2*, a reflective VLA planner that reframes autonomous driving as a joint process of decision making, trajectory drafting, and self-correction, all within a shared discrete token space. A goal-point posterior exposes behavior-level hypotheses before low-level motion generation, masked discrete diffusion drafts editable trajectories in parallel, and AutoEdit rewrites the draft through the same policy without an auxiliary repair network. The central finding of this work is that self-correction in driving planning requires more than a trained editor. Under supervised training alone, AutoEdit exists in the weights but contributes only modestly at inference. Applying reinforcement learning over the complete draft-and-edit rollout changes this: a shared terminal reward co-adapts the drafter and the editor, so that drafts become revisable and edits become reward-seeking. This interaction raises the inference-time AutoEdit gain from +0.3 to +1.9 PDMS and is the primary driver of *ReflectDrive-2*'s 91.0 PDMS on NAVSIM with camera-only input. Under best-of-6 oracle selection, the system reaches 94.8 PDMS, indicating that the goal-point posterior captures a genuinely multi-modal distribution over driving behaviors. We further showed that the decision-draft-reflect structure defines not only a modeling paradigm but also an efficient runtime. Shared-prefix KV cache reuse, Alternating Step Decode, a lightweight action-expert FFN, and fused on-device unmasking together bring the average planner-stack latency to 30.2 ms per frame on NVIDIA Thor, with near-

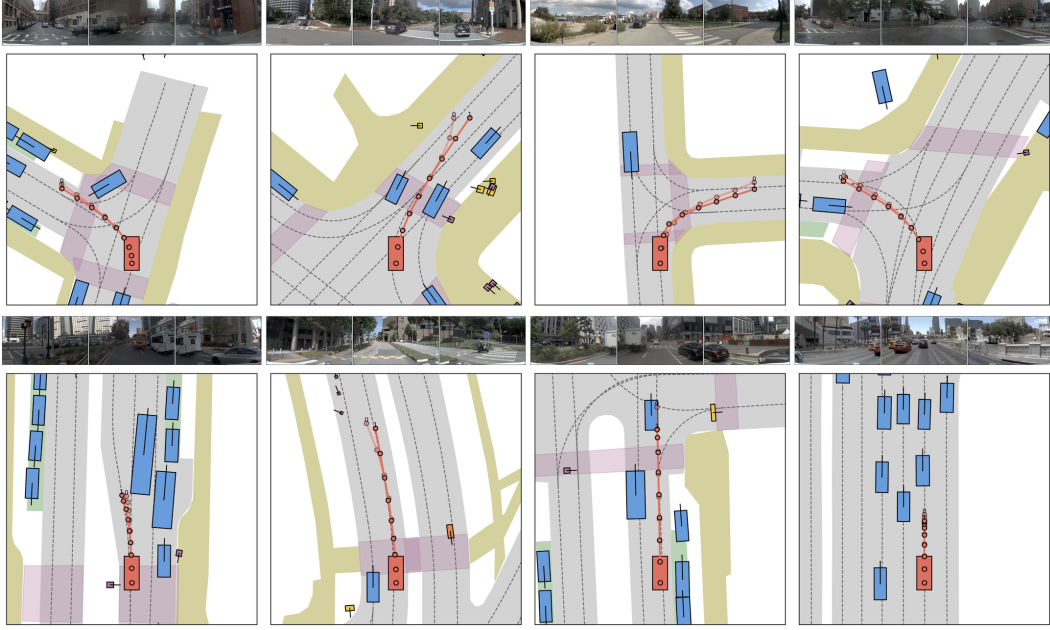


Figure 5: **Inference-time reflection with AutoEdit.** Semi-transparent: initial drafts. Solid: final outputs after AutoEdit. PDMS annotated at trajectory endpoints.

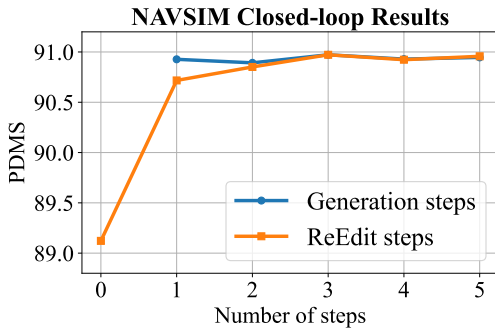


Figure 6: **Sensitivity to diffusion steps.** PDMS with different generation and AutoEdit step counts.

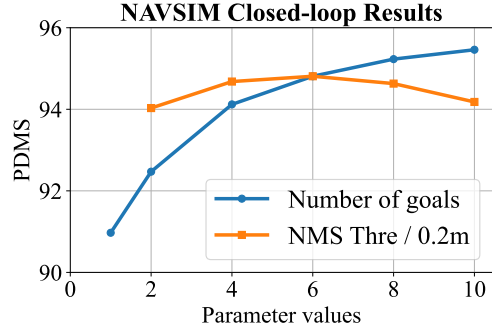


Figure 7: **Sensitivity to goal-proposal parameters.** PDMS vs. number of proposals and NMS threshold.

lossless planning quality. These results suggest that masked discrete diffusion can serve as an editable and deployable foundation for VLA driving policies.

Limitations and future work. ReflectDrive-2 represents trajectories with fixed-resolution BEV coordinate tokens. This choice provides an interpretable and editable action space for masked drafting and AutoEdit, but it also bounds the spatial precision of the generated waypoints by the coordinate-bin size. Future work could improve precision with finer coordinate vocabularies, residual offsets, or hybrid discrete-continuous action heads while retaining token-space editability. Our RL stage currently optimizes a lightweight closed-loop planning score, which is efficient for post-training but remains a proxy for real-world driving objectives. Higher-fidelity interactive simulators and richer safety-oriented rewards may improve alignment, albeit with higher computational cost. In addition, the current AutoEdit perturbations focus on longitudinal progress and lateral heading errors; extending them to interaction-level failures such as yielding timing, cut-in response, and gap selection could improve correction in multi-agent scenes.

References

- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2503.09573>.
- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, volume 34, pages 17981–17993, 2021.
- Mayank Bansal, Alex Krizhevsky, and Abhijit Ogale. ChauffeurNet: Learning to drive by imitating the best and synthesizing the worst. In *Robotics: Science and Systems*, 2019.
- Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, Chengxi Li, Chongxuan Li, Jianguo Li, Zehuan Li, Huabin Liu, Ling Liu, Guoshan Lu, Xiaocheng Lu, Yuxin Ma, Jianfeng Tan, Lanning Wei, Ji-Rong Wen, Yipeng Xing, Xiaolu Zhang, Junbo Zhao, Da Zheng, Jun Zhou, Junlin Zhou, Zhanchao Zhou, Liwang Zhu, and Yihong Zhuang. LLaDA2.0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.
- Tiwei Bie, Maosong Cao, Xiang Cao, Bingsen Chen, Fuyuan Chen, Kun Chen, Lun Du, Daozhao Feng, Haibo Feng, Mingliang Gong, Zhuocheng Gong, Yanmei Gu, Jian Guan, Kaiyuan Guan, Hongliang He, Zenan Huang, Juyong Jiang, Zhonghui Jiang, Zhenzhong Lan, Chengxi Li, Jianguo Li, Zehuan Li, Huabin Liu, Lin Liu, Guoshan Lu, Yuan Lu, Yuxin Ma, Xingyu Mou, Zhenxuan Pan, Kaida Qiu, Yuji Ren, Jianfeng Tan, Yiding Tian, Zian Wang, Lanning Wei, Tao Wu, Yipeng Xing, Wentao Ye, Liangyu Zha, Tianze Zhang, Xiaolu Zhang, Junbo Zhao, Da Zheng, Hao Zhong, Wanli Zhong, Jun Zhou, Junlin Zhou, Liwang Zhu, Muzhi Zhu, Yihong Zhuang, et al. LLaDA2.1: Speeding up text diffusion via token editing. *arXiv preprint arXiv:2602.08676*, 2026.
- Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. In *International Conference on Learning Representations*, 2024.
- Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Praseoon Goyal, Lawrence D. Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, Xin Zhang, Jake Zhao, and Karol Zieba. End to end learning for self-driving cars. *arXiv preprint arXiv:1604.07316*, 2016.
- Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex H. Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. nuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021.
- Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. MaskGIT: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11315–11325, 2022.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, Zehao Yu, Katrin Renz, and Andreas Geiger. TransFuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- Felipe Codevilla, Matthias Müller, Antonio López, Vladlen Koltun, and Alexey Dosovitskiy. End-to-end driving via conditional imitation learning. In *IEEE International Conference on Robotics and Automation*, pages 4693–4700, 2018.
- Chenxu Dang, Sining Ang, Yongkang Li, Haochen Tian, Jie Wang, Guang Li, Hangjun Ye, Jie Ma, Long Chen, and Yan Wang. DriveFine: Refining-augmented masked diffusion VLA for precise and robust driving. *arXiv preprint arXiv:2602.14577*, 2026.
- Daniel Dauner, Matthias Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. NAVSIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *Advances in Neural Information Processing Systems*, 2024.

- Haoyang Fan, Fan Zhu, Changchun Liu, Liangliang Zhang, Li Zhuang, Dong Li, Weichuan Zhu, Jiangtao Hu, Hongye Li, and Qi Kong. Baidu Apollo EM motion planner. *arXiv preprint arXiv:1807.08048*, 2018.
- Xiaoxin Feng, Ziyang Gao, Yuheng Kan, and Wei Wu. SMART: Scalable multi-agent real-time motion generation via next-token prediction. *arXiv preprint arXiv:2405.15677*, 2024.
- Shengchao Hu, Li Chen, Penghao Wu, Hongyang Li, Junchi Yan, and Dacheng Tao. ST-P3: End-to-end vision-based autonomous driving via spatial-temporal feature learning. In *European Conference on Computer Vision*, pages 533–549, 2022.
- Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, Lewei Lu, Xiaosong Jia, Qiang Liu, Jifeng Dai, Yu Qiao, and Hongyang Li. Planning-oriented autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17853–17862, 2023.
- Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, pages 9902–9915, 2022.
- Yatai Ji, Teng Wang, Yuying Ge, Zhiheng Liu, Sidi Yang, Ying Shan, and Ping Luo. From denoising to refining: A corrective framework for vision-language diffusion model. *arXiv preprint arXiv:2510.19871*, 2025.
- Bo Jiang, Shaoyu Chen, Qing Xu, Bencheng Liao, Jiajie Chen, Helong Zhou, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. VAD: Vectorized scene representation for efficient autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8340–8350, 2023a.
- Chiyu Jiang, Andre Cornman, Cheolho Park, Benjamin Sapp, Yin Zhou, and Dragomir Anguelov. MotionDiffuser: Controllable multi-agent motion prediction using diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9644–9653, 2023b.
- Shinpei Kato, Shinpei Tokunaga, Yuya Maruyama, Shigeki Maeda, Manato Hirabayashi, Yuki Kitsukawa, Abraham Monrroy, Tomohito Ando, Yusuke Fujii, and Takuya Azumi. Autoware on board: Enabling autonomous vehicles with embedded systems. In *ACM/IEEE International Conference on Cyber-Physical Systems*, pages 287–296, 2018.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Yingyan Li, Shuyao Shang, Weisong Liu, Bing Zhan, Haochen Wang, Yuqi Wang, Yuntao Chen, Xiaoman Wang, Yasong An, Chufeng Tang, Lu Hou, Lue Fan, and Zhaoxiang Zhang. DriveVLA-W0: World models amplify data scaling law in autonomous driving. *arXiv preprint arXiv:2510.12796*, 2025a.
- Yongkang Li, Kaixin Xiong, Xiangyu Guo, Fang Li, Sixu Yan, Gangwei Xu, Lijun Zhou, Long Chen, Haiyang Sun, Bing Wang, Guang Chen, Hangjun Ye, Wenyu Liu, and Xinggang Wang. ReCogDrive: A reinforced cognitive framework for end-to-end autonomous driving. *arXiv preprint arXiv:2506.08052*, 2025b.
- Zhenxin Li, Kailin Li, Shihao Wang, Shiyi Lan, Zhiding Yu, Yishen Ji, Zhiqi Li, Ziyue Zhu, Jan Kautz, Zuxuan Wu, Yu-Gang Jiang, and José M. Álvarez. Hydra-MDP: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*, 2024.
- Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, and Xinggang Wang. DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12037–12047, 2025. doi: 10.1109/CVPR52734.2025.01124.

- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning*, 2024a.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning*, 2024b. Also referred to as Score Entropy Discrete Diffusion.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy policy optimization. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2409.00588>.
- Chonghao Sima, Katrin Renz, Kashyap Chitta, Li Chen, Hanxue Zhang, Chunguang Xie, Ping Luo, Andreas Geiger, and Hongyang Li. DriveLM: Driving with graph visual question answering. In *European Conference on Computer Vision*, 2024.
- Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, Yuwei Fu, Jing Su, Ge Zhang, Wenhao Huang, Mingxuan Wang, Lin Yan, Xiaoying Jia, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Yonghui Wu, and Hao Zhou. Seed diffusion: A large-scale diffusion language model with high-speed inference. *arXiv preprint arXiv:2508.02193*, 2025.
- Xiaoyu Tian, Junru Gu, Bailin Li, Yicheng Liu, Chenxu Hu, Yang Wang, Kun Zhan, Peng Jia, Xianpeng Lang, and Hang Zhao. DriveVLM: The convergence of autonomous driving and large vision-language models. In *Conference on Robot Learning*, 2024.
- Chenyu Wang, Paria Rashidinejad, DiJia Su, Song Jiang, Sid Wang, Siyan Zhao, Cai Zhou, Shannon Zejiang Shen, Feiyu Chen, Tommi S. Jaakkola, Yuandong Tian, and Bo Liu. SPG: Sandwiched policy gradient for masked diffusion language models. *arXiv preprint arXiv:2510.09541*, 2025a.
- Guanghan Wang, Yair Schiff, Gilad Turok, and Volodymyr Kuleshov. d2: Improved techniques for training reasoning diffusion language models. *arXiv preprint arXiv:2509.21474*, 2025b.
- Yuqing Wen, Hebei Li, Kefan Gu, Yucheng Zhao, Tiancai Wang, and Xiaoyan Sun. LLaDA-VLA: Vision language diffusion action models. *arXiv preprint arXiv:2509.06932*, 2025.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.
- Zebin Xing, Xingyu Zhang, Yang Hu, Bo Jiang, Tong He, Qian Zhang, Xiaoxiao Long, and Wei Yin. GoalFlow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1602–1611, 2025.
- Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *arXiv preprint arXiv:2504.12216*, 2025.
- Yinan Zheng, Tianyi Tan, Bin Huang, Enguang Liu, Ruiming Liang, Jianlin Zhang, Jianwei Cui, Guang Chen, Kun Ma, Hangjun Ye, Long Chen, Ya-Qin Zhang, Xianyu Zhan, and Jingjing Liu. Unleashing the potential of diffusion models for end-to-end autonomous driving. *arXiv preprint arXiv:2602.22801*, 2026.
- Ziyuan Zhong, Davis Rempe, Yuxiao Chen, Boris Ivanovic, Yulong Cao, Danfei Xu, Marco Pavone, and Baishakhi Ray. Language-guided traffic simulation via scene-level diffusion. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 144–177. PMLR, 2023. URL <https://arxiv.org/abs/2306.06344>.
- Zewei Zhou, Tianhui Cai, Seth Z. Zhao, Yun Zhang, Zhiyu Huang, Bolei Zhou, and Jiaqi Ma. AutoVLA: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. In *Advances in Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2506.13757>.